

Feedforward Neural Networks in pytorch



Implementation in Python: pytorch

To implement python DL models we will use the ***PyTorch*** package: this allows you to create, train, and apply several distinct DL models

To install:

pip install torch

To run the examples in this session

pip install torchvision

XNOR dataset

Jupyter Notebook:
Xnor-pytorch.ypnnb

XNOR
dataset

Dataset that we used before just with 4 exemples
Nonlinear dataset used to illustrate
Just training examples

```
X = torch.tensor([
    [0., 0.],
    [0., 1.],
    [1., 0.],
    [1., 1.]
])
y = torch.tensor([
    [1.],
    [0.],
    [0.],
    [1.]
])
```

Loading the data into Torch
tensors

DNNs for XNOR

```
class XNORNet(nn.Module):
    def __init__(self, hidden = 2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, hidden), # hidden layer - 4 nodes
            nn.Sigmoid(),         # nonlinearity (important!)
            nn.Linear(hidden, 1), # output layer - 1 node
            nn.Sigmoid()          # output as probability
        )

    def forward(self, x):
        return self.net(x)

model = XNORNet(4)
```

Defining the model –
Sequential class to create
feedforward network

One hidden layer (with
number of nodes as
parameter)

Sigmoid activation function in
hidden and output layers

DNNs for XNOR

Binary cross-entropy loss (Logistic regression)

Adam algorithm for training

```
# Loss and optimizer
```

```
criterion = nn.BCELoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.05)
```

```
epochs = 100
```

```
for epoch in range(epochs):
```

```
    # forward
```

```
    y_pred = model(X)
```

```
    loss = criterion(y_pred, y)
```

```
    # backward
```

```
    optimizer.zero_grad()
```

```
    loss.backward()
```

```
    optimizer.step()
```

```
...
```

Training cycle

In each epoch:

first forward, then backward

```
with torch.no_grad():
```

```
    preds = model(X)
```

```
    ...
```

Predicting for the training cases

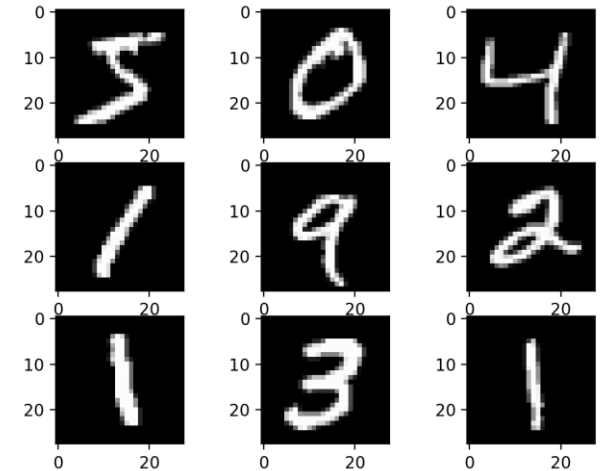
MNIST dataset

Jupyter Notebook:
torch-example-mnist.ipynb

MNIST dataset

Dataset with images of digits in 28x28 pixels grid with 784 features representing pixels in grey scale 0 to 255. Output: class representing the digit (10 classes, digits 0-9). Available as a keras dataset. 60K samples – training set; 10K images – test

This dataset will be used here in the following example, vectorizing images to a 1D vector with 784 values as inputs for each image, and defining a multi-class classification problem with 10 possible outputs



DNNs for MNIST

Loading the training and test data from a
Torch dataset

```
# Transformations to apply to the data
transform = transforms.Compose([
    transforms.ToTensor() # Convert image to PyTorch Tensor
])

# Download and load the training data
trainset = torchvision.datasets.MNIST(root='./data', train=True, download=True,
                                       transform=transform)
trainloader = DataLoader(trainset, batch_size=64, shuffle=True, num_workers=2)

# Download and load the test data
testset = torchvision.datasets.MNIST(root='./data', train=False, download=True,
                                      transform=transform)
testloader = DataLoader(testset, batch_size=1000, shuffle=False, num_workers=2)
```

DNNs for MNIST

```
class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128) # Input layer -> Hidden layer 1
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64) # Hidden layer 1 -> Hidden layer 2
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10) # Hidden layer 2 -> Output layer

    def forward(self, x):
        x = self.flatten(x) # Flatten the image
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x) # Raw scores (logits)
        return x

model = SimpleMLP().to(device)
```

Defining the model and the topology of the NN creating the computation graph functionally

Flattening the data (images -> vector for each example)

2 hidden layers with 128 and 64 neurons

ReLU activation functions

Output layer with 10 neurons (number of classes)

DNNs for MNIST

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
num_epochs = 10
```

```
for epoch in range(num_epochs):
```

```
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
```

```
        inputs, labels = data[0].to(device), data[1].to(device)
```

```
        optimizer.zero_grad()
```

```
        # Forward pass
```

```
        outputs = model(inputs)
```

```
        loss = criterion(outputs, labels)
```

```
        # Backward pass and optimize
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
...
```

Loss function – cross entropy
(for multiclass)

Training cycle similar to
previous example

MNIST dataset – CNN model



Jupyter Notebook:
CNN_mnist.ipynb

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), # (B, 32, 28, 28)
            nn.ReLU(),
            nn.MaxPool2d(2),              # (B, 32, 14, 14)
            nn.Conv2d(32, 64, 3, padding=1), # (B, 64, 14, 14)
            nn.ReLU(),
            nn.MaxPool2d(2)               # (B, 64, 7, 7)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)           # 10 classes
        )
    def forward(self, x):
        return self.fc(self.conv(x))
```

32 channels; window 3 x 3

Alternating conv layers with pooling

64 channels

Layers to allow classification:

Flatten – tensor 3D to 1D

Dense – fully connected feedforward layers

Output layer – 10 neurons

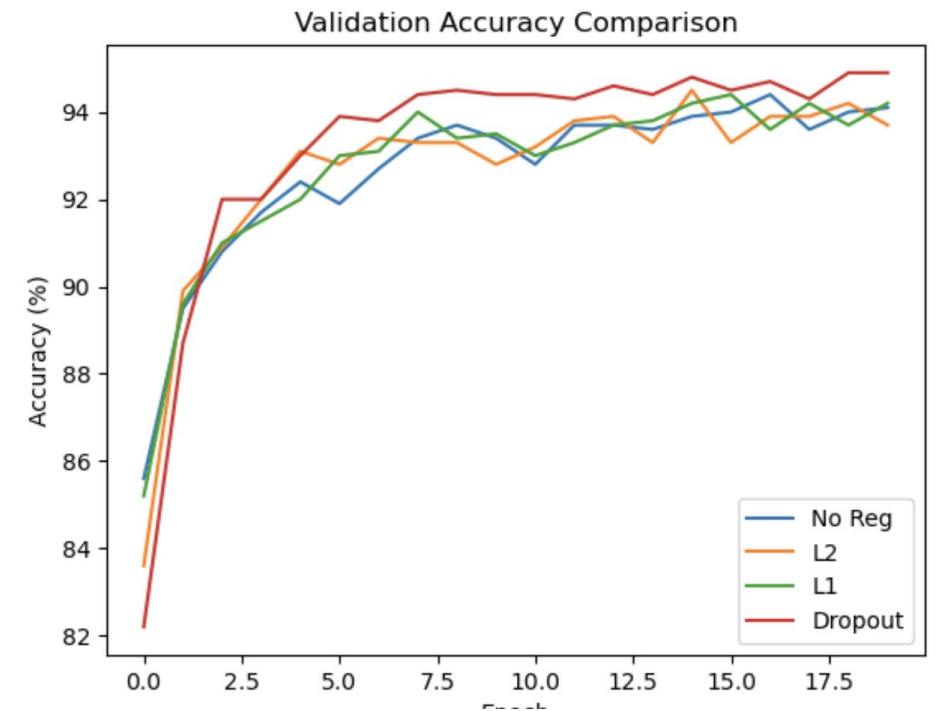
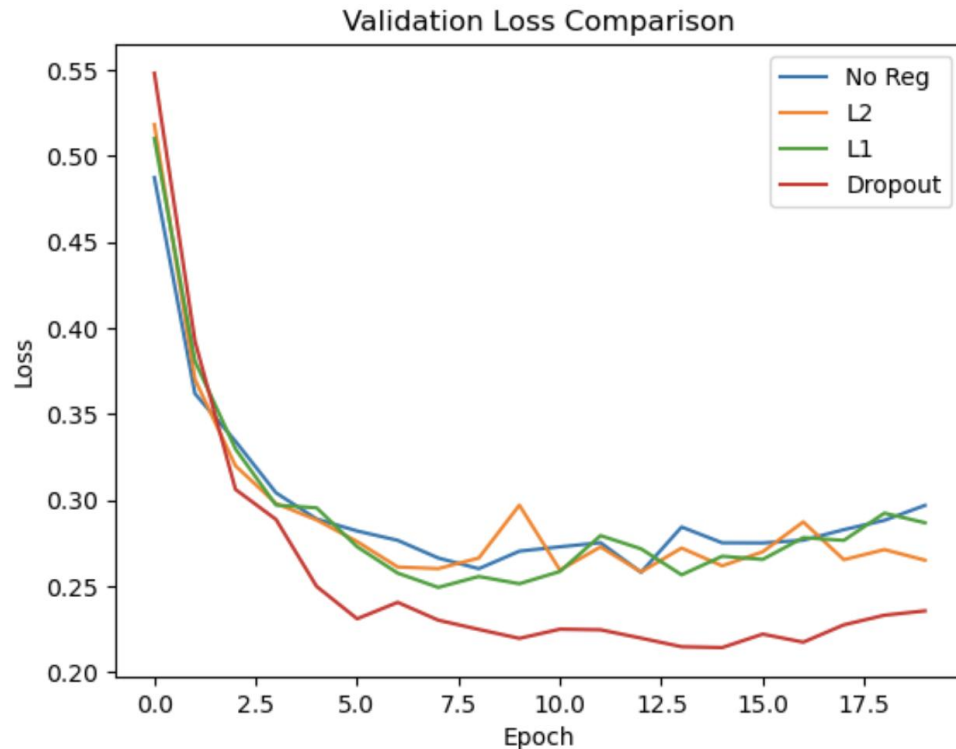
Training and evaluation are the same
as FFNNs

Overfitting in pytorch

Jupyter Notebook:
`mnist_regularization.ipynb`

MNIST

- No regularization
- Dropout
- L1
- L2



Boston housing dataset - regression

Jupyter Notebook:
boston-torch.ipynb

Boston
dataset

Predicting the average price of houses in Boston (in the 1970s)
Inputs: several features from the house
404 training examples
102 test examples

Example shows how to load data from a scikit-learn dataset

Example of a regression problem

Check the notebook for details

DNN for Boston housing

```
class HousingNet(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1) # no sigmoid → regression output
        )

    def forward(self, x):
        return self.net(x)

model = HousingNet(X_train.shape[1])
```

Main difference in the model is the use of a Linear output without an activation function

Loss function: MSE

```
criterion = nn.MSELoss()
optimizer = optim.Adam(...)
```

Exercise



- Load the dataset HAR - *Human Activity Recognition using Smartphones* dataset
- Dataset description:

The experiments have been carried out with a group of 30 volunteers. Each person performed six activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING) wearing a smartphone. Using its embedded accelerometer and gyroscope, we captured 3-axial linear acceleration and 3-axial angular velocity. The experiments have been video-recorded to label the data manually.

- **Variables:** For each record in the dataset it is provided:
 - A 561-feature vector with time and frequency domain variables.
 - Its activity label.
 - An identifier of the subject who carried out the experiment.
- More details:

<https://archive.ics.uci.edu/ml/datasets/human+activity+recognition+using+smartphones>

The exercise aims to create Deep Neural Networks for this dataset (the data is provided in a CSV file preprocessed and loading is already implemented in the notebook); compare results with best SVM model provided